



36. Decision Tree Regressor

Can we “branch” numerical values?

Previous Section:

 [35. Ridge & Lasso Regression](#)

Contents:

[1. Decision Tree Regressor – The Overview](#)

[2. Decision Tree Regressor – Standard Deviation Reduction](#)

[3. Decision Tree Regressor with Python](#)

[Summary](#)

[References](#)

Every concept in Machine Learning comes with a first impression (*This is not the first time that I have said this*). And honestly, Decision Trees feel like one of the most intuitive models you’ll ever see. “Ask a question → follow a path → get an answer”. Simple. Almost too simple.

That’s why most explanations stop there. They show you how the tree splits, how it makes decisions, and you walk away thinking: **“Alright... I get it.”**

But here’s the problem. That understanding is incomplete. Because what you usually see is **Decision Trees for classification**—where everything feels natural: **Yes / No; True / False; Class A / Class B**. Clean. Discrete. Easy to branch.

But now let’s ask a slightly uncomfortable question:

┃ ***What happens when the output is not a category... but a number?***

Not "Is this spam?". But "How much does this house cost?"; Not "Cat or dog?". But "What is the temperature tomorrow?". Now things change. Because numbers don't fit neatly into boxes. They don't naturally split into clean labels.

And yet, Decision Trees still work. So the real question is no longer: "How does a tree make decisions?" → But rather: ***"How does a tree predict continuous values... using splits that look inherently discrete?"***

That's where the **Decision Tree Regressor** begins.

1. Decision Tree Regressor – The Overview

Alright, now we're stepping into something slightly different. Most of the time, when people introduce Decision Trees, they show you classification. Clean labels. Clear decisions. Everything fits nicely into boxes. But reality doesn't always give you boxes. Sometimes, it gives you numbers. Not "yes" or "no". But *"how much?"*. And surprisingly, Decision Trees handle that just fine.

Think of it this way. A Decision Tree Regressor doesn't try to *classify* things. It tries to **estimate a value**. So instead of ending with: "This is Class A" → It ends with something like: "This house is worth... around 250,000" → Same structure. Different purpose.

The process still starts at the root—just like before. We ask a question about a feature: ***"Is size > 2000 sqft?"*** → ***"Is age < 10 years?"***

And based on the answer, we split the dataset into branches. Then we do it again. And again. And again. Each split pushes the data into smaller, more specific groups. Until eventually, we reach a leaf. But here's the key difference: ***That leaf doesn't contain a class. It contains a number—usually the average of all values in that group.***

So when a new data point comes in? We don't "decide" its class. We simply follow the path... and **read off the number at the end**.

Now, let's talk about how the tree *decides* where to split. If you remember ID3—what we've been calling *Entropy & Information Gain*—it works beautifully for classification.

But here's the issue: Entropy doesn't make sense for numbers. You can't really measure "purity" the same way when your output is continuous. So we need a different idea. Instead of asking: "How pure is this group?". We ask: **"How similar are these numbers?"**. And that's where **standard deviation** comes in.

Think about a group of values. If they're all very close to each other... low standard deviation → very "clean" group; If they're spread all over the place... → high standard deviation → messy group

So the goal becomes simple: ***"At every split... reduce the standard deviation as much as possible"***. This idea is called **Standard Deviation Reduction**. It replaces Information Gain as the decision rule.

So the tree grows like this:

- Start with all data → measure how scattered the values are
- Try a split → check if the groups become less scattered
- Pick the split that reduces the scatter the most
- And repeat. Top-down. Greedy. No going back.

And if you zoom out, the entire algorithm is doing one thing: It keeps dividing the data until each group becomes as **numerically consistent** as possible. Because in regression, "purity" doesn't mean same label. It means: ***"These values are so similar... I can confidently represent them with a single number."***

And that is the core idea behind a Decision Tree Regressor.

2. Decision Tree Regressor – Standard Deviation Reduction

Alright, now we stop talking in theory. Let's actually *build* a Decision Tree Regressor. And this time, we're not guessing. We're going to follow the numbers—step by step—like a proper investigation.

Here's what we're working with:

Outlook	Temp	Humidity	Windy	Hours Played
Rainy	Hot	High	False	26
Rainy	Hot	High	True	30
Overcast	Hot	High	False	46
Sunny	Mild	High	False	46
Sunny	Cool	Normal	False	62
Sunny	Cool	Normal	True	23
Overcast	Cool	Normal	True	43

Outlook	Temp	Humidity	Windy	Hours Played
Rainy	Mild	High	False	36
Rainy	Cool	Normal	False	38
Sunny	Mild	Normal	False	48
Rainy	Mild	Normal	True	48
Overcast	Mild	High	True	52
Overcast	Hot	Normal	False	44
Sunny	Mild	High	True	30

Number of samples: $n = 14$

Step 1: Initial Target Statistics

Before we split anything... we need to understand the data as a whole. Think of this as: "How messy are the numbers right now?"

- **Mean (Hours Played):** $\bar{y} = 40.857$
- **Standard Deviation:** $s = 10.911$
- **Coefficient of Variation (CV):**

$$V = \frac{s}{\bar{y}} \times 100 = \frac{10.911}{40.857} \times 100 = 26.7\%$$



Important detail, don't skip this: We are using **sample standard deviation** (divide by $n - 1$). Why? Because in Machine Learning, we almost never have the full population. We learn from samples. So this 10.911 becomes our baseline. Every split we make from now on must answer one question: **"Did we reduce this number?"**

Step 2: Standard Deviation for Each Feature

Now we test each feature. We simulate splitting, without actually committing yet.

The formula:

$$Std(Target, Feature) = \sum_{c \in Feature} P(c) \cdot Std(c)$$

Where:

- $P(c) = \frac{n_c}{n}$
- $Std(c)$ = standard deviation inside each group

Feature: Outlook

Category	n	Std
Rainy	5	8.41
Sunny	5	15.46
Overcast	4	4.03

$$Std(Hours, Outlook) = \frac{5}{14}(8.41) + \frac{5}{14}(15.46) + \frac{4}{14}(4.03) = 9.67$$

Feature: Temp

Category	n	Std
Hot	4	9.98
Mild	6	8.45
Cool	4	16.09

$$Std(Hours, Temp) = \frac{4}{14}(9.98) + \frac{6}{14}(8.45) + \frac{4}{14}(16.09) = 11.07$$

Feature: Humidity

Category	n	Std
High	7	10.00
Normal	7	11.78

$$Std(Hours, Humidity) = \frac{7}{14}(10.00) + \frac{7}{14}(11.78) = 10.89$$

Feature: Windy

Category	n	Std
True	6	11.60
False	8	10.47

$$Std(Hours, Windy) = \frac{6}{14}(11.60) + \frac{8}{14}(10.47) = 10.95$$

Step 3: Standard Deviation Reduction (SDR)

Now we compare everything to the baseline.

$$SDR = Std(Target) - Std(Target, Feature)$$

Given: $Std(Target) = 10.911$

Feature	$Std(Hours, Feature)$	SDR
Outlook	9.67	1.24
Temp	11.07	-0.15
Humidity	10.89	0.021
Windy	10.95	-0.038

Now pause... and observe.

- **Outlook** → **best reduction**
- **Humidity** → **barely helps**
- **Temp & Windy** → **actually make things worse**

So the choice is obvious: **Root Node = Outlook**

Step 4a: Split by Outlook

We now divide the dataset.

- **Outlook = Rainy**

Temp	Humidity	Windy	Hours
Hot	High	F	26
Hot	High	T	30
Mild	High	F	36
Cool	Normal	F	38

Temp	Humidity	Windy	Hours
Mild	Normal	T	48

- **Outlook = Sunny**

Temp	Humidity	Windy	Hours
Mild	High	F	46
Cool	Normal	F	62
Cool	Normal	T	23
Mild	Normal	F	48
Mild	High	T	30

- **Outlook = Overcast**

Temp	Humidity	Windy	Hours
Hot	High	F	46
Cool	Normal	T	43
Mild	High	T	52
Hot	Normal	F	44

Step 4b: Coefficient of Variation Threshold

We introduce a stopping rule: **CV Threshold = 20%**

Group	Std	Mean	CV
Rainy	8.41	35.6	23.6%
Sunny	15.46	41.8	36.9%
Overcast	4.03	46.25	8.71%

Key Decision

- Overcast → $CV < 20\%$ → **STOP**
- Rainy & Sunny → still messy → **continue**

So: **Leaf Node (Overcast) = 46.25**

Recursive Split: Outlook = Sunny

Stats: Mean = 41.8; Std = 15.46; CV = 37% → continue

- **Try Feature: Temp**

Category	n	Std
Mild	3	9.865
Cool	2	27.577

$$Std = \frac{3}{5}(9.865) + \frac{2}{5}(27.577) = 16.9498$$

$$SDR = 15.46 - 16.9498 = -1.48$$

- **Try Feature: Humidity**

Category	n	Std
High	2	11.31
Normal	3	19.75

$$Std = \frac{2}{5}(11.31) + \frac{3}{5}(19.75) = 16.374$$

$$SDR = 15.46 - 16.374 = -0.914$$

- **Try Feature: Windy**

Category	n	Std
True	2	4.94
False	3	8.71

$$Std = \frac{2}{5}(4.94) + \frac{3}{5}(8.71) = 7.202$$

$$SDR = 15.46 - 7.202 = 8.258$$

→ **Final Decision: Best Split = Windy**

Recursive Split: Outlook = Rainy

Stats: Mean = 35.6; Std = 8.414; CV = 23.6% → continue

- **Try Feature: Temp**

Category	n	Std
Mild	2	8.48
Cool	1	0
Hot	2	2.82

$$Std = \frac{2}{5}(8.48) + \frac{1}{5}(0) + \frac{2}{5}(2.82) = 4.52$$

$$SDR = 8.414 - 4.52 = 3.894$$

- **Try Feature: Humidity**

Category	n	Std
High	3	5.03
Normal	2	7.07

$$Std = \frac{3}{5}(5.03) + \frac{2}{5}(7.07) = 5.846$$

$$SDR = 8.414 - 5.846 = 2.568$$

- **Try Feature: Windy**

Category	n	Std
True	2	12.72
False	3	6.429

$$Std = \frac{2}{5}(12.72) + \frac{3}{5}(6.429) = 8.9454$$

$$SDR = 8.414 - 8.9454 = -0.5314$$

Final Decision → Best Split = Temp

Where We Are Now

If you step back and look at the process. This isn't random at all. The tree is doing one thing—over and over: Measure variability. Try a split. Keep the one that reduces it the most And slowly, the chaos turns into structure.

And that's the real beauty here: We didn't "tell" the model what matters. We just gave it a rule: **"Make the numbers less messy."** → And from that single idea, a full Decision Tree emerges.

Final Recursive Splitting and Leaf Nodes

We're almost there. At this point, the tree already has structure. But structure alone isn't enough. We need to make sure that **every branch is "stable enough" to stop**.

And remember the rule we set: **CV Threshold = 20%** → So now, we go branch by branch. No shortcuts.

- **Branch 1: Outlook = Sunny → Split by Windy**

Temp	Humidity	Windy	Hours
Mild	High	False	46
Cool	Normal	False	62
Cool	Normal	True	23
Mild	Normal	False	48
Mild	High	True	30

Case 1: Windy = True → Values: 23, 30

- **Standard Deviation:** $Std = 4.94$
- **Mean:** $\bar{y} = 26.5$
- **Coefficient of Variation:**

$$CV = \frac{4.94}{26.5} = 18.64\% < 20\%$$

Variability is low enough → **STOP** → **Leaf Prediction = 26.5**

Case 2: Windy = False → Values: 46, 62, 48

- **Standard Deviation:** $Std = 8.71$
- **Mean:** $\bar{y} = 52$
- **Coefficient of Variation:**

$$CV = \frac{8.71}{52} = 16.75\% < 20\%$$

Stable → **STOP** → **Leaf Prediction = 52**

- **Branch 2: Outlook = Rainy → Split by Temp**

Temp	Humidity	Windy	Hours
Hot	High	False	26
Hot	High	True	30
Mild	High	False	36
Cool	Normal	False	38
Mild	Normal	True	48

Case 1: Temp = Hot → Values: 26, 30

- $Std = 2.82; Mean = 28$

$$CV = \frac{2.82}{28} = 10.07\% < 20\%$$

Stable → **STOP** → **Leaf Prediction = 28**

Case 2: Temp = Cool → Values: 38

- $Std = 0; Mean = 38$

$$CV = 0 < 20\%$$

Perfectly homogeneous → **STOP immediately** → **Leaf Prediction = 38**

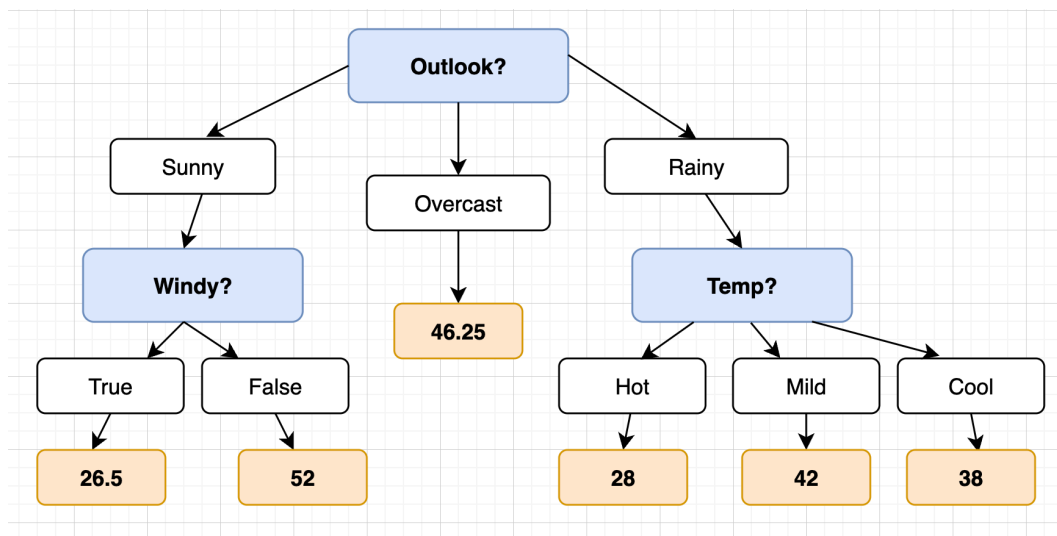
Case 3: Temp = Mild → Values: 36, 48

- $Std = 8.48; Mean = 42$

$$CV = \frac{8.48}{42} \approx 20.2\% \approx 20\%$$

Now this is interesting. It's right on the edge. And this is where judgment comes in. Instead of forcing another split for almost no gain, we accept it → **STOP** → **Leaf Prediction = 42**

Final Decision Tree Structure (Conceptual) → If you step back... the entire tree now looks like this:



At this stage, something important has happened. We didn't just split randomly. We enforced a rule:

Every leaf must be “numerically stable”

And we measured that using CV. So the tree stopped not because it *ran out of features*. But because **there was nothing meaningful left to reduce**.

Sample Prediction

Let's test it with a **new instance**:

Outlook	Temp	Humidity	Windy
Sunny	Hot	High	True

Path: *Outlook = Sunny* → *Windy = True* → Reach leaf: Prediction ≈ 26.5

Now, let's be honest. CV is great for learning. But in real-world Machine Learning? We don't rely on it alone. Because it can be unstable when sample size is small; data is noisy; values fluctuate slightly

So in practice, we often use:

- Minimum samples per leaf
- Maximum tree depth
- Minimum variance reduction
- Cost-complexity pruning

For example: *"Stop if a node has fewer than 2–3 samples"* → This simple rule is often more reliable than CV.

If you really understand what just happened, then you understand Decision Tree Regression. Because the tree is not trying to be "smart". It's doing something much simpler:

Reduce variability → Group similar numbers → Represent each group with one value

And repeat, until nothing meaningful changes. That's it. No magic. Just structure emerging from numbers.

3. Decision Tree Regressor with Python

Decision Tree Regressor with Python is straightforward using the Scikit-Learn library

```
import pandas as pd

data = {"Outlook": [0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1, 1, 2],
        "Temp": [0, 0, 0, 1, 2, 2, 2, 1, 2, 1, 1, 1, 0, 1],
        "Humidity": [0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0],
        "Windy": [0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1],
        "Hours Played": [26, 30, 46, 46, 62, 23, 43, 36, 38, 48, 48, 52, 44, 30]}

df = pd.DataFrame(data)
```

```
from sklearn.model_selection import train_test_split

target = "Hours Played"
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
# Features and Target
X_train = train_df[df.columns.drop(target)]
y_train = train_df[target]
```

```
X_test = test_df[df.columns.drop(target)]
y_test = test_df[target]
```

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

model = DecisionTreeRegressor(max_depth=4, random_state=42)
model.fit(X_train, y_train)
# Make prediction
y_pred = model.predict(X_test)
print(f"MSE: {mean_squared_error(y_test, y_pred)}")
print(f"MAE: {mean_absolute_error(y_test, y_pred)}")
print(f"R^2 Score: {r2_score(y_test, y_pred)}")
```

MSE: 161.66666666666666

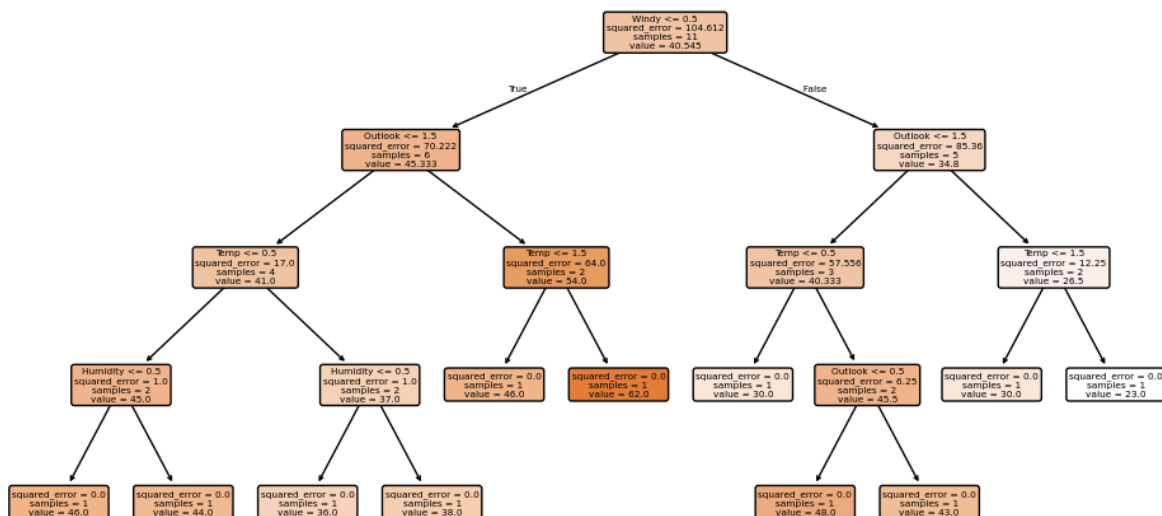
MAE: 10.333333333333334

R^2 Score: -0.2372448979591837

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# Plot Decision Tree
plt.figure(figsize=(12, 6))
plot_tree(model, feature_names=df.columns.drop(target).tolist(),
          filled=True, rounded=True)
plt.title("Decision Tree Regressor")
plt.show()
```

Decision Tree Regressor



Summary

Decision Tree Regressor is technically a different model—but not a different idea. At its core, it is still the same Decision Tree you already understand: A structure of questions. A process of splitting data. A path from root to leaf. Nothing about that changes.

What *does* change is the goal. Instead of separating data into classes, we are now trying to **approximate numbers**. And that small shift changes everything:

- Leaves no longer store labels → they store **values (usually averages)**
- “Purity” is no longer about class consistency → it becomes **numerical similarity**
- Information Gain is no longer meaningful → we replace it with **variance / standard deviation reduction**

So if Classification Trees ask: “How can I separate different groups?”; Then Regression Trees ask: **“How can I make each group as numerically consistent as possible?”**

And in the end, the philosophy remains the same:

- ***Split the data → Reduce uncertainty → Repeat recursively***

Until what remains is simple enough to represent with a single answer.

At the end of the day, Decision Tree Regressor is a “new model” in terms of purpose. But structurally? It is still the same tree—just solving a different kind of problem.

References

http://www.saedsayad.com/decision_tree_reg.htm

<https://www.geeksforgeeks.org/machine-learning/python-decision-tree-regression-using-sklearn/>

<https://www.youtube.com/watch?v=UhY5vPfQlrA>

Next Section:

 [37. Softmax Regression](#)